

ICHIGO CHESS

High-Level Architecture (HLD) and Low-Level Design (LLD)

A web-development-oriented, end-to-end technical report

System under study

Browser frontend: HTML + CSS + browser-native ES modules

Application server: Node.js + Express

Persistence: SQLite through better-sqlite3

Chess rules: chess.js on the server

Engine integration: opening book + UCI engine endpoints

Puzzle sources: local database, Chess.com, engine analysis, Python ML scoring

Prepared for a software developer learning web-system architecture

25 July 2026

Document scope, evidence, and certainty

This report explains both the architecture of the refactored Ichigo frontend and the runtime behavior of the supplied Node/Express server. It is intentionally written for a developer who understands software design but is new to browser/server web development. It therefore explains the network, browser, DOM, HTTP, Express, event-loop, and module-loading concepts that connect the files together.

Evidence boundary

The frontend module structure and class responsibilities are verified from the refactored code and the later board-interaction update. The supplied server.js text begins partway through the file, so the visible game, puzzle, Chess.com, and app.listen code is verified, while some startup middleware and endpoints such as /game/new, /bookmove, /bestmove, /puzzles/daily, and /puzzles/random are described from observed runtime behavior and the frontend contracts.

Label used in this report	Meaning
Verified	Directly visible in the supplied code or demonstrated by successful HTTP requests.
Inferred	Standard Node/Express/browser behavior required to explain an observed result, but the exact line was outside the supplied server excerpt.
Recommended	A design improvement or safe extension, not necessarily present in the current code.

Executive summary

Ichigo is a same-origin, server-backed web application with two browser pages: Play and Puzzles. The browser is responsible for rendering, interaction, transient UI state, and HTTP requests. The Node/Express server is responsible for receiving those HTTP requests, validating and persisting games, calling the chess engine, sourcing puzzles, and integrating with Chess.com and the Python scoring pipeline.

The refactored frontend follows a layered architecture with feature-oriented packaging. Each page has a composition root that constructs a state aggregate, passive view, API gateway, board adapter, and controller. Controllers orchestrate use cases. Domain objects represent browser-side state. Infrastructure objects hide HTTP details. The backend uses Express route handlers as application entry points, chess.js as the authoritative rule engine, better-sqlite3 as the persistence adapter, and helper functions/processes for engine and ML work.

The single most important boundary

The frontend ChessPosition is a visual and state projection, not the authoritative legal-move engine. For Play, the server accepts or rejects moves. After acceptance, the frontend reconciles itself to the server-returned FEN and move history. This avoids two competing chess engines.

Table of contents

Part I - Web runtime and big-picture architecture

1. What npm run dev actually starts
2. Runtime actors, processes, and communication boundaries
3. High-level architecture and deployment topology
4. Browser boot and frontend composition

Part II - Frontend component architecture

5. Presentation, application, domain, and infrastructure layers
6. Play feature design
7. Puzzle feature design
8. Shared services and adapters

9. State ownership, source of truth, and concurrency

Part III - Backend architecture

10. Express routing, middleware, and request dispatch

11. Game persistence and chess validation

12. Engine, opening-book, and puzzle-generation subsystems

Part IV - End-to-end execution examples

13. Initial page load

14. New game creation

15. Player move, rejection rollback, and engine reply

16. Daily puzzle and personalized puzzle flows

17. Failure, 404, CSP, and debugging flows

Part V - Low-level design patterns

18. Pattern inventory and detailed mapping

19. API contracts and DTOs

20. Modifying, extending, testing, and scaling the system

21. Design critique, risks, and roadmap

Appendices - file map, glossary, checklists, and sequence summaries

Part I - Web runtime and big-picture architecture

1. What npm run dev actually starts

Your package script is effectively `PORT=8080 node server.js`. The command is short, but it activates several layers of software.

1.1 npm is a command dispatcher, not the web server

When you run `npm run dev`, npm reads `package.json`, finds the `dev` entry under `scripts`, performs environment-variable substitution through the shell, and launches the configured child process. npm remains the parent process, but Node is the process that executes the server code.

```
Terminal shell
├─ npm process
│   └─ executes script: PORT=8080 node server.js
│       └─ Node.js process loads server.js
```

Why npm run server.js failed

`npm run` expects the name of a script key, not a filename. `node server.js` executes a file directly. `npm run dev` executes the script named `dev` from `package.json`.

1.2 Node loads and evaluates server.js

1. Node creates a JavaScript runtime: heap, call stack, module loader, event loop, timers, and native I/O bindings.
2. Node evaluates `server.js` from top to bottom. Imports/requires are resolved, constants are initialized, database connections and engine processes may be created, middleware and routes are registered, and helper functions become available.
3. Calling `app.get(...)` or `app.post(...)` registers route matchers and handler functions in the Express application. These calls do not each open a network socket.
4. Calling `app.listen(PORT, HOST, callback)` asks Node to create one HTTP server and bind its underlying TCP socket to `0.0.0.0:8080`.
5. The callback prints HTTP on `0.0.0.0:8080` after the operating system confirms that the port is bound.
6. The initial JavaScript execution finishes, but the process stays alive because the HTTP socket, engine child processes, timers, and other handles are active.

1.3 What is actually listening?

Listener or callback	Where it runs	What wakes it up
TCP/HTTP listener created by <code>app.listen</code>	Node process + operating-system socket	A network connection or HTTP request reaches port 8080.
Express middleware and route handlers	Node call stack	The HTTP server passes a parsed request into the Express application stack.
Browser DOM event listeners	Browser main thread	Clicks, pointer movement, key presses, resize events, and form changes.
<code>chessboard.js</code> callbacks	Browser main thread	Board drag start, drop, and snap-end lifecycle events.
Fetch promise continuations	Browser microtask queue	An HTTP response, abort, or network failure completes a fetch request.
Timers	Browser or Node event loop	<code>setTimeout</code> / <code>setInterval</code> deadlines become ready.

Important distinction

The Express route functions are not independent servers. There is one listening HTTP server. Express is a dispatcher that selects one or more registered middleware/route functions for each request.

1.4 The event loop in practical terms

Node does not dedicate one JavaScript thread to each browser. Instead, the operating system and Node native libraries monitor sockets and other I/O. When work becomes ready, Node queues a callback. The event loop repeatedly takes ready callbacks and executes their JavaScript on the main thread. This is why asynchronous I/O scales well, but long CPU-bound JavaScript blocks every request.

```

Request arrives
↓
OS/network stack receives bytes
↓
Node HTTP parser creates req/res objects
↓
Express middleware/route handler runs
↓
Handler may start async I/O and yield
↓
Event loop handles other work
↓
I/O completes; promise continuation is queued
↓
Handler resumes and sends res.json(...)
    
```

CPU-bound warning

Engine analysis, PGN replay across many games, and synchronous SQLite work can increase response latency. External engine work should run in child processes; large analysis jobs may eventually need a worker queue rather than staying inside a request lifecycle.

2. Runtime actors, processes, and communication boundaries

2.1 The complete runtime topology

```

User
| pointer, keyboard, navigation
▼
Browser process
├── HTML document / DOM
├── CSS layout and painting
├── Play or Puzzle ES-module object graph
├── jQuery + chessboard.js
├── fetch('/...')
│   └── HTTP over localhost:8080
│       ▼
Node.js process
├── Express application and route stack
├── better-sqlite3 connection
├── chess.js rule reconstruction
├── opening-book / engine coordination
├── Chess.com HTTP client
├── Python ML scoring process
│   ├── SQLite database file
│   ├── UCI engine child process(es)
│   └── api.chess.com
    
```

2.2 Process versus module versus object

Concept	Ichigo example	Meaning
Process	Browser, Node server, chess engine, Python scorer	An operating-system execution container with its own memory.
Module/file	play-controller.mjs, server.js	A unit of source code loaded into a process.
Object instance	new PlayController(...), Express app	Runtime state and behavior created from modules/classes.
Callback/handler	handleDrop, app.post route function	A function invoked later when an event or request occurs.
Protocol	HTTP/JSON, UCI, PGN, FEN	A structured agreement that allows components/processes to communicate.

2.3 Same-origin communication

The browser loads both static assets and APIs from localhost:8080. Because the page and API share scheme, host, and port, relative URLs such as /game/new are same-origin. The browser therefore sends them to the same Express server without needing a CORS policy or proxy.

```
Page origin:    http://localhost:8080
Relative fetch: /game/new
Resolved URL:   http://localhost:8080/game/new
```

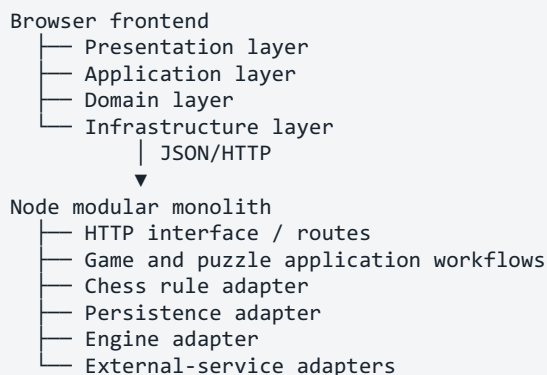
Why opening file:// failed conceptually

A file:// page has no normal HTTP origin and cannot naturally resolve the same server-relative API lifecycle. Serving through Express makes browser modules and same-origin HTTP requests behave consistently.

3. High-level architecture and deployment topology

3.1 Architectural style

At the highest level, Ichigo is a client-server application. Within the frontend, it uses a layered architecture and feature-oriented packaging. Within the backend, it is currently a modular monolith: one Node process contains HTTP routing, application workflows, persistence access, engine integration, Chess.com integration, and puzzle-generation coordination.



3.2 Quality attributes driving the design

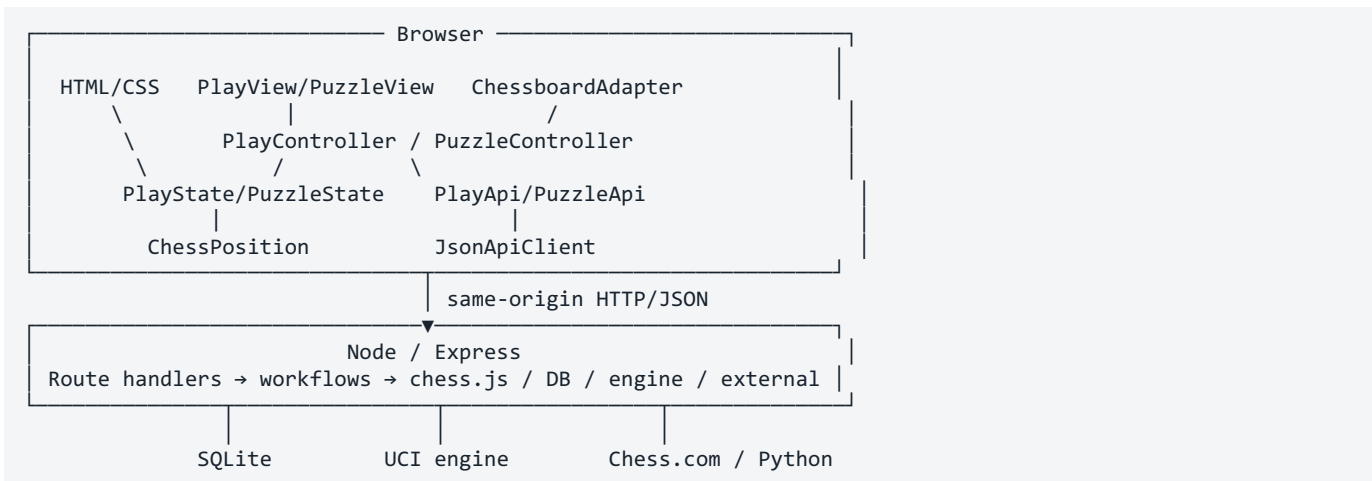
Quality attribute	Design response
Correctness	Server-authoritative Play validation; authoritative FEN reconciliation; puzzle projection verification.

Quality attribute	Design response
Modifiability	Play, Puzzles, and shared concerns are separated into modules with explicit dependencies.
Testability	State, controllers, gateways, and domain logic can be tested with substitutes instead of a real browser or server.
Understandability	Each file has a dominant responsibility and a named architectural pattern.
Reliability	Memento rollback, request-version guards, AbortController cancellation, and input locks.
Security	External text is rendered with textContent/DOM creation rather than innerHTML.
Responsiveness	Optimistic local moves and avoidance of redundant chessboard redraws.

3.3 System boundaries and sources of truth

Concern	Authoritative owner	Frontend responsibility
Legal Play moves	Server chess.js reconstruction	Project move immediately, then accept authoritative response or roll back.
Persisted game history	SQLite via server routes	Display a local copy and reconcile to returned history.
Checkmate/draw/result	Server	Render result and disable interaction.
Engine/book decision	Backend engine/book endpoints	Request and display the returned move.
Puzzle solution	Backend puzzle DTO	Allow only the expected UCI sequence.
Board pixels and interaction state	Browser	Render, highlight, select, drag, click, resize.
Displayed clocks	Browser CountdownClock	Maintain responsive local countdown and send values to engine endpoint.

3.4 HLD component diagram



4. Browser boot and frontend composition

4.1 Initial document request and asset discovery

When the user navigates to <http://localhost:8080/>, the browser sends GET /. Express serves public/index.html, either through express.static or an explicit sendFile route. The browser parses the HTML from top to bottom and discovers referenced CSS, fonts, classic scripts, and the module entry point.

1. HTML parser creates DOM nodes for the header, controls, board container, clocks, move list, log, and buttons.
2. The browser fetches chessboard.js CSS, base.css, and play.css and computes the cascade.
3. jQuery is loaded as a classic script and creates global window.\$ / window.jQuery.
4. chessboard.js is loaded as a classic script and creates global window.Chessboard.
5. The module script ./js/play/play.mjs is fetched. Browser-native module loading recursively fetches its imports.
6. After dependencies are evaluated, play.mjs constructs the frontend object graph and calls controller.initialize().

4.2 Why play.mjs and puzzles.mjs are composition roots

A composition root is the one place where concrete implementations are created and connected. It should know which classes are used, while business classes should only know the interfaces of their collaborators. This prevents object construction from being scattered across controllers and views.

```
const state = new PlayState();
const view = new PlayView(document);
const api = new PlayApi(new JsonApiClient());
const board = new ChessboardAdapter(...);
const clock = new CountdownClock(...);
controller = new PlayController({ state, api, view, boardAdapter: board, clock });
controller.initialize();
```

This is dependency injection without a framework. The controller does not import and instantiate its own API client, board, view, or clock. Tests can therefore pass fakes with the same methods.

4.3 Browser event registration

Source event	Registered by	Target method
New Game button click	PlayView.bind	PlayController.startNewGame
Flip button click	PlayView.bind	PlayController.flipBoard
Board drag start	Chessboard config in play.mjs	PlayController.canDrag
Board drop	Chessboard config in play.mjs	PlayController.handleDrop
Board snap end	Chessboard config in play.mjs	PlayController.handleSnapEnd
Square click	ChessboardAdapter delegated click listener	PlayController.handleSquareClick or PuzzleController.handleSquareClick
Window resize	Composition root	ChessboardAdapter.resize
Daily/Random/From Games controls	PuzzleView.bind	PuzzleController source methods

Part II - Frontend component architecture

5. Layer-by-layer responsibilities

5.1 Presentation layer

The presentation layer contains HTML, CSS, passive view classes, and the chessboard adapter. It owns browser-specific details: DOM element IDs, text insertion, CSS class toggling, board library calls, and accessibility attributes. It should not decide whether an engine move is needed or which endpoint to call.

Component	Primary role	Inputs	Outputs / side effects
index.html / puzzles.html	Static document structure	Browser navigation	DOM skeleton and asset references
base.css / play.css /	Visual presentation	DOM classes and state	Layout, color, animation, responsiveness

Component	Primary role	Inputs	Outputs / side effects
puzzles.css		classes	
PlayView	Play DOM facade	Render models and handler callbacks	Text/classes/elements updated; user values safely read
PuzzleView	Puzzle DOM facade	Puzzle metadata, progress, game list	DOM updates and event forwarding
ChessboardAdapter	Third-party board isolation	ChessPosition and highlight commands	chessboard.js calls and square CSS classes

5.2 Application layer

Controllers implement complete use cases and sequencing. They combine user intent, state transitions, domain behavior, I/O, cancellation, rollback, and rendering. They are application services because their logic describes workflows rather than one domain entity.

- **PlayController:** new game, click/drag move, server submission, reconciliation, engine request, hint, finish, timeout, rollback, render coordination.
- **PuzzleController:** source switching, puzzle normalization, validation, click/drag attempt, automatic reply, auto-play, recent-game loading, personalized generation, stale-response protection.

5.3 Domain layer

The domain layer represents state in browser-independent objects. ChessPosition understands FEN, squares, pieces, UCI syntax, board projection, castling projection, en-passant metadata, promotion, and move counters. PlayState and PuzzleState define the consistency boundaries for their pages.

Domain model limitation

ChessPosition applies trusted UCI moves structurally. It is not a full move generator and does not prove that a bishop moved diagonally or that a king stayed out of check. That legality boundary intentionally remains on the server.

5.4 Infrastructure layer

Infrastructure modules translate application intent into external communication. JsonApiClient owns fetch, JSON encoding/decoding, and HTTP error conversion. PlayApi and PuzzleApi own endpoint paths and request DTO shapes. Controllers call domain-oriented methods such as createGame or getDailyPuzzle instead of constructing URLs.

Controller intent	Infrastructure translation
-----	-----
api.createGame(...)	→ POST /game/new
api.submitMove(id, uci)	→ POST /game/:id/move
api.getBestMove(...)	→ POST /bestmove
api.getDailyPuzzle()	→ GET /puzzles/daily
api.getRecentGames(user)	→ GET /chesscom/:username/games/recent

6. Play feature design

6.1 PlayState as Aggregate Root

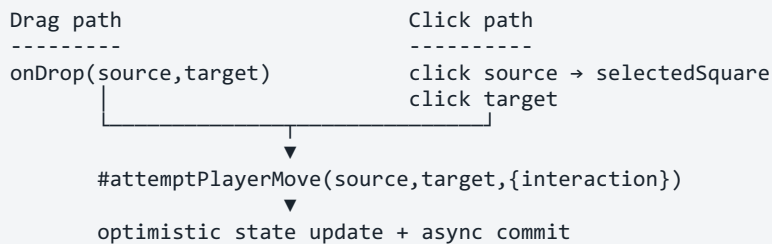
PlayState groups all mutable client state that must change coherently: current ChessPosition, move list, game ID, lifecycle, human color, orientation, selected square, hint move, locks, result, PGN, and a monotonically increasing sessionVersion. The controller modifies this aggregate; views receive data to render but do not mutate it directly.

State field	Meaning	Who changes it
position	Current browser projection of the board and FEN metadata	PlayController, then server reconciliation

State field	Meaning	Who changes it
moves	UCI history displayed and sent to book/engine endpoints	PlayController; replaced by authoritative response when available
lifecycle	idle, active, or finished	PlayState.start/finish/resetToldle
gameId	Server database game identifier	PlayState.start after POST /game/new
sessionVersion	Generation token for stale async work	invalidateSession, start, finish
inputLocked	Prevents overlapping moves while validation/engine work is pending	PlayController
selectedSquare	Click-to-move selection state	PlayController.handleSquareClick
hintMove	UCI move highlighted as a hint	PlayController.requestHint

6.2 Click and drag interaction as one use case

Drag-and-drop and click-to-move are two input mechanisms that converge on the same private move-attempt function. This is important: different UI gestures must not create different chess behavior.



During a drag, `chessboard.js` has already moved the piece in its DOM. The controller therefore renders metadata and highlights without calling `board.position` immediately. `onSnapEnd` performs the later synchronization. For click movement there is no native drag projection, so the controller explicitly synchronizes the board. This distinction removed the perceptible double-redraw delay.

6.3 Optimistic update and Memento rollback

The user expects a piece to move immediately, but the server must validate legality. The controller therefore uses an optimistic transaction: snapshot, mutate locally, send request, then commit by reconciliation or roll back by restoring the snapshot.

```

Memento = {
  position: cloned ChessPosition,
  moves: copied array,
  hintMove,
  selectedSquare,
  inputLocked
}
  
```

1. Create a Memento before changing state.
2. Stop the local clock so network latency is not charged to a side during validation.
3. Apply the UCI move to the local projection and append it to moves.
4. Lock input and render immediately.
5. POST the move through `PlayApi` and `JsonApiClient`.
6. On success, replace the local projection/history with authoritative server values when provided.
7. On rejection or transport failure, restore the Memento, unlock input, restart the clock, log the error, and redraw.

6.4 Session versions and abortable engine requests

Asynchronous responses can arrive after the user starts another game or ends the current one. A `sessionVersion` token protects against stale continuation. Engine and hint requests also use `AbortController` so a new session can actively cancel the fetch.

```
const sessionVersion = state.sessionVersion;
const response = await api.getBestMove(...);
if (state.sessionVersion !== sessionVersion) return; // stale response ignored
```

Why this matters

Without generation tokens, an old New Game response could overwrite a newer game, or an engine result from the prior position could move a piece in the current game.

6.5 PlayView as Passive View

`PlayView` centralizes DOM lookup and rendering. It binds browser events to callbacks supplied by the controller, reads simple control values, and renders moves, clocks, status, logs, and banners. It does not call APIs or decide workflow order. This keeps DOM-specific code out of the controller and makes the controller testable with a plain JavaScript fake view.

6.6 PlayApi and JsonApiClient

`PlayApi` provides a vocabulary that matches the Play use cases. `JsonApiClient` provides transport behavior shared by Play and Puzzles. The browser fetch function must be bound to `globalThis` when stored as an object property; otherwise some environments produce an illegal invocation. The installed correction uses `fetchImpl.bind(globalThis)`.

Method	HTTP request	Expected response use
<code>createGame</code>	POST /game/new {time_control}	Returns <code>gameId</code> used for all persisted move routes
<code>submitMove</code>	POST /game/:id/move {uci}	Authoritative FEN/history/result
<code>finishGame</code>	POST /game/:id/finish {result}	Result, reason if supported, and PGN
<code>getBookMove</code>	POST /bookmove {moves}	Optional bookmove
<code>getBestMove</code>	POST /bestmove {moves, timing, depth}	bestmove and optional raw engine output

7. Puzzle feature design

7.1 PuzzleState and source modes

`PuzzleState` owns the current FEN, solution array, solution index, orientation, selection, last move, active source, personalized puzzle collection, loading state, and validation error. It supports Daily, Random, and From Games sources without changing the core move-execution behavior.

7.2 Exact-solution validation

A puzzle is not a free-play board. The controller compares the attempted source and destination against the next expected UCI move. A correct user move advances the index, then schedules the opponent reply from the same solution sequence. An incorrect move produces feedback and leaves the position unchanged.

```
expected = solution[index]
parse expected into from/to

if attempted from/to !== expected from/to:
  show "Incorrect – try again"
else:
```

```

apply expected UCI
index += 1
schedule next solution move if puzzle remains

```

7.3 Projection verification before interaction

Before a puzzle becomes playable, PuzzleController clones a ChessPosition from the supplied FEN and applies every solution move. If any move cannot be structurally projected, the puzzle is displayed with a validation error but interaction is blocked. This converts malformed backend data into a visible contract failure rather than a corrupted UI.

7.4 Public and personalized DTO normalization

Public puzzle endpoints use fen and moves. Personalized endpoints may return moves, solutionMoves, or a fallback single uci field. The controller normalizes these variants at the boundary, so the rest of the puzzle state always uses one solution array. This is an Anti-Corruption Layer in a lightweight form: inconsistent external schemas are translated into one internal model.

7.5 Stale load protection

PuzzleController increments loadVersion whenever a new load begins. Every awaited operation checks the captured version before applying results. This protects rapid source switching and the two-stage recent-games workflow, where recent games may load before personalized analysis finishes.

8. Shared services and adapters

Module	Pattern	Responsibility	Key design value
chess-position.mjs	Domain Model	FEN/UCI projection, board metadata, cloning	One shared representation for both pages
chessboard-adapter.mjs	Adapter	Wrap global chessboard.js and DOM square classes	Third-party dependency isolation
countdown-clock.mjs	Service Object	Drift-aware countdown and timeout callback	Reusable time behavior
api-client.mjs	Infrastructure Service	fetch, JSON, HTTP errors, abort propagation	Consistent transport behavior
errors.mjs	Error hierarchy	ApplicationError, ApiError, ChessDataError	Clear failure categories
dom.mjs	Utility / construction helper	Required element lookup and safe DOM creation	Fail fast and avoid innerHTML

9. State ownership, source of truth, and concurrency

9.1 Why state ownership matters

A UI becomes unpredictable when the board library, DOM, controller, state object, and server each independently claim to own the current position. The refactor gives the controller authority to mutate the state aggregate, treats the board library as a renderer, and treats the server as the final authority for legal persisted Play state.

```

Server-authoritative state
  ↓ reconciliation
PlayState / PuzzleState
  ↓ render model
Views + ChessboardAdapter
  ↓ pixels and controls
User

```

9.2 Concurrency controls used

Mechanism	Problem solved
inputLocked	Prevents a second move while the first is awaiting server validation or engine work.
pendingMoveSubmissions	Prevents finishing a game while a move write is still in flight.
sessionVersion	Ignores stale work from a previous Play session.
startRequestVersion	Only the newest New Game response may initialize state.
loadVersion	Only the newest puzzle/game-analysis request may update PuzzleState.
AbortController	Actively cancels engine/hint HTTP requests when no longer relevant.
Timer cancellation	Stops auto-reply/auto-play callbacks when source or puzzle changes.

Part III - Backend architecture

10. Express routing, middleware, and request dispatch

10.1 Express application structure

Express app is a JavaScript object that behaves like a request handler. Registration calls such as `app.get(path, handler)` and `app.post(path, handler)` append route layers to its internal stack. Middleware such as JSON parsing and static-file serving is also placed in that ordered stack.

Typical server startup order (some early lines inferred)

```
-----
const app = express();
app.use(express.json());
app.use(express.static(publicDirectory));
register game, engine, puzzle, and Chess.com routes
app.listen(PORT, HOST, callback);
```

Order is behavior

Express walks middleware and routes in registration order. A broad route can shadow a more specific later route. The visible `/game/:id` route appeared before `/game/:id.pgn`, so `/game/12.pgn` could be consumed as `id="12.pgn"`. The hardened router registers the PGN route first.

10.2 Request dispatch algorithm

1. The Node HTTP server receives method, URL, headers, and body bytes.
2. Express creates req and res wrappers and begins at the first stack layer.
3. Each middleware checks whether it applies. It may modify req/res, terminate the response, or call `next()`.
4. A route matches both method and path pattern. For `/game/:id/move`, Express extracts `req.params.id`.
5. The handler reads `req.body`, `req.params`, and `req.query`, calls collaborators, and eventually calls `res.json`, `res.send`, or `res.sendFile`.
6. Once a response is sent, Express/Node serializes headers and body onto the socket. The browser fetch promise resolves when the response arrives.

10.3 Static files and API routes share one server

Request	Likely/verified handler type	Response
GET /	Static middleware or explicit file route	public/index.html

Request	Likely/verified handler type	Response
GET /css/base.css	Static middleware	CSS bytes
GET /js/play/play.mjs	Static middleware	JavaScript module
GET /puzzles	Verified sendFile route	public/puzzles.html
POST /game/new	API route	JSON gameId
GET /puzzles/daily	API route	JSON puzzle DTO
GET /favicon.ico	No matching static file/route	Express 404 HTML response

11. Game persistence and chess validation

11.1 Persisted-game move route

The visible POST /game/:id/move route reconstructs the game from stored UCI history, asks chess.js to apply the new move, computes terminal status, updates SQLite, and returns an authoritative DTO.

```

HTTP route
  → validate id and UCI
  → SELECT game row
  → parse stored UCI history
  → gameFromMoves(history)
  → chess.js game.move({from,to,promotion})
  → compute checkmate/stalemate/draw conditions
  → UPDATE games
  → return {moves, fen, over, reason, result, pgn}

```

11.2 Why reconstruct from history?

The database stores a move sequence rather than a live chess.js object. A JavaScript object cannot be persisted directly across server restarts. gameFromMoves creates a fresh game and replays the UCI history, restoring the position and rule history needed for validation. This is a form of event-sourced reconstruction, although the broader system is not a formal event-sourcing architecture.

11.3 Persistence adapter

better-sqlite3 provides synchronous prepared statements. Synchronous calls simplify transaction semantics and error handling, but they execute on the Node main thread. For small local game writes this is reasonable; heavy queries or large concurrent workloads would eventually require careful indexing, batching, or a different deployment model.

Operation	Data purpose
UPDATE games SET moves=? WHERE id=?	Persist accepted nonterminal move history
UPDATE games SET moves=?, result=?, pgn=? WHERE id=?	Atomically persist a terminal move in the hardened design
SELECT moves, pgn, time_control, result FROM games WHERE id=?	Retrieve or regenerate PGN
SELECT ... FROM games ORDER BY id DESC LIMIT 50	Recent-game listing

11.4 Backend hardening patterns

The optional createGameRouter extraction applies Router Factory and Dependency Injection patterns. It receives Express, DB, game reconstruction, chess.js compatibility resolution, and PGN generation as collaborators. It also centralizes UCI/result/id normalization and wraps related writes in better-sqlite3 transactions.

- Reject non-string or malformed UCI before calling slice.

- Catch modern chess.js illegal-move exceptions.
- Validate game IDs as safe positive integers.
- Restrict results to *, 1-0, 0-1, or 1/2-1/2.
- Register /game/:id.pgn before /game/:id.
- Use one atomic terminal update.

12. Engine, opening-book, and puzzle-generation subsystems

12.1 Play engine workflow

The browser first asks /bookmove for a move matching the UCI history. Book lookup is optional; failure is logged and the controller falls back to /bestmove. The best-move request includes move history, move time or depth, and both displayed clocks. The backend coordinates the UCI engine and returns a bestmove string.

```
PlayController.requestEngineMove
  1. POST /bookmove
  2. if no bookmove → POST /bestmove
  3. project returned UCI locally
  4. POST same UCI to /game/:id/move
  5. reconcile to authoritative FEN/history
```

12.2 Chess.com recent-game adapter

GET /chesscom/:username/games/recent normalizes the username, fetches the user archive list from the Chess.com public API, walks monthly archives from newest to oldest, and returns at most the requested number of normalized game DTOs. fetchJsonWithTimeout uses AbortController and a timeout to prevent an external request from hanging indefinitely.

12.3 Personalized puzzle pipeline

```
Chess.com PGN games
  ↓ loadPgnCompat
Verbose chess.js history and replay
  ↓ candidate positions for the selected player
Pre-move engine evaluation + PV
Post-played-move engine evaluation
  ↓ feature calculation
Eval-gap threshold filtering
  ↓
Python model scoring
  ↓
Sort, cap, normalize solution DTO
  ↓
POST /puzzles/from-user-ml response
```

The pipeline coordinates multiple representations: PGN for entire games, FEN for individual positions, UCI for moves, centipawns/mate values for engine evaluations, and a model score for ranking. The HTTP boundary should normalize these into a stable Puzzle DTO.

Current design debt

The visible server contains duplicated evaluation pipelines and a fallback that stores centipawn eval_gap_cp in ml_score, even though the normal score appears probability-like. These should be separated into ranking_score plus ranking_score_type or normalized to one unit.

Part IV - End-to-end execution examples

13. End-to-end example: loading the Play page

1. The user enters `http://localhost:8080/` in the address bar.
2. The browser resolves `localhost`, opens a TCP connection to port 8080, and sends `GET /`.
3. Node receives the request and passes it to Express.
4. Static middleware or the root route returns `public/index.html`.
5. The browser parses HTML and sends additional `GET` requests for Google fonts, chessboard CSS, `base.css`, `play.css`, `jQuery`, `chessboard.js`, `play.mjs`, and every imported module.
6. CSS is parsed into style rules. The HTML becomes the DOM. `jQuery` and `Chessboard` become globals.
7. `play.mjs` evaluates, constructs `PlayState`, `PlayView`, `PlayApi/JsonApiClient`, `ChessboardAdapter`, `CountdownClock`, and `PlayController`.
8. `controller.initialize` binds buttons, keyboard shortcuts, and square clicks; disables the inactive board; and renders the initial state.
9. The page is now interactive, but no persisted game exists until `New Game` is clicked.

```
GET /
  → index.html
    → GET /css/base.css
    → GET /css/play.css
    → GET /chessboardjs...css
    → GET jQuery
    → GET chessboard.js
    → GET /js/play/play.mjs
      → imported ES-module graph
        → construct object graph
        → initialize listeners
```

14. End-to-end example: creating a new game

1. The browser dispatches a click event to the `New Game` button.
2. `PlayView` forwards the event to the callback registered by `PlayController.initialize`.
3. `PlayController.startNewGame` increments `startRequestVersion`, aborts engine work, stops the clock, invalidates the old session, locks the board, and renders `Starting....`
4. `PlayApi.createGame` calls `JsonApiClient.post` with `/game/new` and `{time_control: "300+0"}`.
5. `JsonApiClient` serializes JSON, adds `Accept` and `Content-Type` headers, and calls bound `fetch`.
6. The browser sends `POST /game/new` to `localhost:8080`.
7. Express JSON middleware parses the body; the route creates a database row and responds `{gameId: 142}`.
8. The `fetch` promise resolves; `JsonApiClient` parses JSON; `PlayApi` verifies `gameId`.
9. The controller checks that this is still the newest request, starts `PlayState`, resets/starts the clock, enables/orients the board, and renders.
10. If the human selected `Black`, the controller immediately enters the engine workflow.

```
User click
  → PlayView event callback
  → PlayController.startNewGame
  → PlayApi.createGame
  → JsonApiClient.post
  → HTTP POST /game/new
  → Express handler
  → SQLite INSERT
  ← {gameId}
  ← Promise resolution
```


→ PlayState.start
→ board + clock + DOM render

15. End-to-end example: player move accepted

15.1 Browser-side immediate path

1. The user drags e2 to e4 or clicks e2 then e4.
2. ChessboardAdapter/chessboard.js forwards the gesture to PlayController.
3. The controller verifies active session, unlocked input, human turn, and ownership of the source piece.
4. It infers queen promotion when a pawn reaches rank 1 or 8 and builds UCI e2e4.
5. It creates a Memento, stops the clock, applies the UCI to ChessPosition, appends the move, clears selection/hint, and locks input.
6. The UI updates immediately. During drag the board library already moved the piece; during click the adapter is explicitly synchronized.

15.2 Network and server path

1. The controller asynchronously calls POST /game/142/move with {uci: "e2e4"}.
2. Express matches /game/:id/move and extracts id=142.
3. The route validates the UCI and loads stored move history.
4. gameFromMoves replays history into chess.js.
5. chess.js validates e2→e4 and mutates the reconstructed game.
6. The route computes terminal conditions, persists the new history, and sends authoritative FEN and moves.

Response example

```
{
  "ok": true,
  "moves": "e2e4",
  "fen": "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1",
  "over": false,
  "reason": null,
  "result": "*",
  "pgn": null
}
```

15.3 Client reconciliation and engine handoff

1. PlayController parses the returned FEN into a new ChessPosition.
2. It parses and replaces the local move list with the authoritative history.
3. It unlocks input, restarts the clock, and renders.
4. Because sideToMove is now Black and the human is White, it calls requestEngineMove.
5. The engine workflow gets a book move or best move, applies it optimistically, persists it through the same move endpoint, and reconciles again.

15.4 End-to-end example: rejected move and rollback

The frontend projection may structurally apply a move that is illegal under full chess rules. For example, a pinned piece could expose its king. The server returns HTTP 422. JsonApiClient converts that non-2xx response into ApiError. The controller restores the Memento, unlocks input, restarts the clock, logs the rejection, and renders the original position.

Optimistic local state

↓ POST move



Server rejects 422 {error:"illegal move"}

```

| ApiError
▼
state.restore(memento)
├── original ChessPosition
├── original move list
├── original highlights
└── original input state
    
```

16. End-to-end example: daily puzzle

1. The Puzzles page bootstraps through puzzles.mjs and PuzzleController.initialize.
2. Initialization triggers loadDaily.
3. PuzzleApi.getDailyPuzzle sends GET /puzzles/daily.
4. The server returns a DTO containing id/title, FEN, UCI solution moves, rating, and themes.
5. PuzzleController validates required fields, normalizes every UCI, and verifies that the entire solution can be projected from the FEN.
6. PuzzleState.loadPuzzle stores the normalized puzzle and computes orientation.
7. PuzzleView renders metadata and progress; ChessboardAdapter renders the board.
8. The user clicks or drags the expected first move. The controller compares exact from/to squares.
9. If correct, it applies the expected move, increments index, and schedules the opponent reply. If incorrect, it shows feedback without changing the position.
10. The puzzle is solved when index reaches solution.length.

16.1 End-to-end example: From Games

1. The user selects From Games and enters a Chess.com username.
2. PuzzleController increments loadVersion and requests recent games.
3. The server calls Chess.com archive endpoints and returns normalized games.
4. PuzzleView safely constructs one button per game using createElement/textContent.
5. In parallel sequence, the controller requests /puzzles/from-user-ml.
6. The server analyzes selected PGNs, calls engine evaluations, computes features, invokes Python scoring, ranks candidates, and returns normalized puzzle objects.
7. The controller ignores the result if a newer request changed loadVersion.
8. Otherwise it stores the puzzle collection and displays the first puzzle. Prev/Next change only the local collection index.

17. End-to-end example: 404, favicon, and CSP diagnostics

A browser automatically requests /favicon.ico when no favicon is declared. Because no matching static file or route exists, Express returns its 404 response. The 404 response carried Content-Security-Policy: default-src none, but the successful application pages did not. Therefore the CSP message belonged to the error document, not to the functioning Play or Puzzle JavaScript.

Observation	Interpretation
POST /game/new succeeds with curl	Server process and API route are reachable.
GET /puzzles/daily succeeds with curl	Puzzle API is healthy.
GET /js/play/play.js or .mjs succeeds	Static module serving is healthy.
GET /favicon.ico returns 404	Missing cosmetic resource; unrelated to game API failure.
Cannot GET /game	No GET /game route; address bar always performs GET.

Observation	Interpretation
Generic frontend “Network request failed” while curl succeeds	Browser/client wrapper issue, wrong origin, abort, extension, or hidden exception. In this project the stored unbound fetch caused the issue.

Part V - Low-level design patterns

18. Pattern inventory

Pattern	Concrete implementation	Why it is used
Layered Architecture	Presentation, Application, Domain, Infrastructure	Controls dependency direction and separates reasons to change.
Feature-oriented packaging	js/play, js/puzzles, js/shared	Keeps use-case code together while sharing stable primitives.
Composition Root	play.mjs, puzzles.mjs	Centralizes object construction and dependency wiring.
Dependency Injection	Controller constructor parameters	Enables explicit dependencies and test substitutes.
Controller / Application Service	PlayController, PuzzleController	Orchestrates end-to-end workflows.
Passive View	PlayView, PuzzleView	Encapsulates DOM without owning business sequencing.
Domain Model	ChessPosition	Encapsulates FEN/UCI position behavior and invariants.
Aggregate Root	PlayState, PuzzleState	Defines page-level consistency boundary.
Repository / Gateway	PlayApi, PuzzleApi	Hides HTTP endpoints behind application vocabulary.
Adapter	ChessboardAdapter	Translates domain representation to third-party library calls.
Memento	PlayState.createMemento/restore	Atomic optimistic rollback.
Service Object	JsonApiClient, CountdownClock	Reusable behavior with focused state.
Error hierarchy	ApplicationError, ApiError, ChessDataError	Typed failure propagation and rendering.
Generation token	sessionVersion, loadVersion	Prevents stale asynchronous writes.
Router Factory	createGameRouter	Extracts and injects backend route dependencies.
Transaction Script / Unit of Work boundary	better-sqlite3 transaction closures	Atomically persists related fields.

18.1 Controller versus View versus State

Question	Controller	View	State/Domain
What should happen next?	Yes	No	No
How is a DOM element updated?	No	Yes	No
What endpoint should be called?	Chooses API method	No	No
What is the current game/puzzle state?	Reads/mutates	Receives render data	Owns data
How is FEN parsed or UCI applied?	Delegates	No	ChessPosition
How is a third-party board rendered?	Issues command	No	ChessboardAdapter translates

18.2 Repository/Gateway versus API client

PlayApi and PuzzleApi know the application-specific endpoints and DTO shapes. JsonApiClient knows generic HTTP/JSON behavior. This two-level split avoids putting `/game/:id/move` knowledge in a generic transport class and avoids duplicating fetch/error code in every gateway method.

```

PlayController
  ↓ createGame({timeControl})
PlayApi
  ↓ post('/game/new', {time_control})
JsonApiClient
  ↓ fetch + JSON + HTTP error conversion
Browser networking
  
```

18.3 Adapter pattern in two directions

ChessboardAdapter translates the internal ChessPosition into the chessboard.js position object and turns domain highlights into library-specific CSS class operations. The backend chess.js helpers perform a different adapter role: they normalize differences between modern and legacy method names such as isCheckmate versus in_checkmate and loadPgn versus load_pgn.

18.4 Memento versus full event sourcing

The frontend Memento is a short-lived rollback snapshot for one optimistic command. The backend stored UCI list resembles an event log because a game can be reconstructed by replay. However, the system does not implement the full operational model of event sourcing: there is no general event store, projection infrastructure, event versioning, or event-driven integration.

18.5 Patterns intentionally not added

The frontend does not yet need an event bus, command hierarchy, Redux-style global store, abstract factories, or a dependency-injection container. Adding them would increase indirection without solving a current scale problem. The direct constructor-injection design keeps the call graph visible to a developer learning the system.

19. API contracts and DTOs

Method and path	Request	Core response
POST /game/new	{time_control}	{gameId}
POST /game/:id/move	{uci}	{ok,moves,fen,over,reason,result,pgn}
POST /game/:id/finish	{result}	{ok,result,reason?,pgn}
GET /game/:id	Path id	Persisted game row
GET /game/:id.pgn	Path id	text/plain PGN
GET /games	None	Recent game summaries
POST /bookmove	{moves}	{bookmove?}
POST /bestmove	{moves,movetimeMs,depth,wtime,btime,winc,binc}	{bestmove,raw?}
GET /puzzles/daily	None	{id,fen,moves,rating,themes,...}
GET /puzzles/random	None	Puzzle DTO
GET /chesscom/:username/games/recent?limit=N	Path/query	{username,archives,games}
POST /puzzles/from-game	{pgn,username,maxPuzzles}	{ok,count,puzzles}
POST /puzzles/from-user-ml	{username,maxGames,maxPuzzles,movetimeMs}	{ok,username,count,puzzles}

19.1 Error semantics

Category	Example	Frontend handling
Transport failure	Connection refused, illegal invocation, DNS failure	JsonApiClient throws ApiError status 0; controller logs/rolls back.

Category	Example	Frontend handling
Abort	AbortController cancels stale engine request	AbortError is usually ignored as expected cancellation.
HTTP validation	400 bad UCI, invalid ID	ApiError with status and server message.
HTTP domain rejection	422 illegal move	Restore Memento and show move rejected.
Conflict	409 corrupt game history	Surface error; server data requires repair.
Not found	404 game/user/puzzle	Display meaningful status; do not confuse with network failure.
Contract failure	2xx response with invalid FEN or missing gameld	Log/throw contract-specific error; avoid silent corruption.
Domain projection error	Malformed FEN/UCI	ChessDataError; block puzzle interaction or local move.

Part VI - Modifying, extending, testing, and scaling

20. How to approach changes safely

20.1 Change by responsibility, not by visual proximity

Desired change	Primary file(s)	Likely secondary impact
Change colors/layout	css/base.css, play.css, puzzles.css	HTML classes only if structure changes
Add a Play button	index.html + PlayView.bind + PlayController method	Possibly PlayApi if server work is needed
Add a new API endpoint call	PlayApi/PuzzleApi	Controller workflow, API tests, server route
Change board library	ChessboardAdapter + composition-root config	Minimal controller/domain impact
Add a state field	PlayState/PuzzleState	Controller transitions, render methods, tests
Change FEN/UCI rules	ChessPosition	Shared tests and both page flows
Change game persistence	server route/repository code	API response contract and migration
Add a puzzle source	PuzzleSource, PuzzleApi, PuzzleController, PuzzleView, HTML	Server endpoint and DTO normalization

20.2 Example: add a Saved Puzzles source

1. Define SAVED in PuzzleSource.
2. Add a Saved button to puzzles.html and a corresponding element in PuzzleView.
3. Extend PuzzleView.bind with onSave.
4. Add PuzzleApi.getSavedPuzzles or getSavedPuzzle.
5. Add a PuzzleController.loadSaved workflow that uses loadVersion and normalizes the DTO.
6. Add GET /puzzles/saved on the server and persistence queries.
7. Keep the resulting internal model {fen,moves,metadata} so existing solving behavior is unchanged.
8. Add controller, gateway, and contract tests.

20.3 Example: add authentication and multiple users

Authentication changes the HLD because game and puzzle data must be associated with an identity. Recommended additions are an authentication middleware boundary, a user/session context on req, ownership checks in game routes, user_id foreign keys, and frontend login/session state. API gateways remain useful, but requests may need credentials and 401 handling. Do not put authorization only in the browser; the server must enforce ownership.

20.4 Example: replace chessboard.js

Because controllers interact through ChessboardAdapter, a replacement board library should implement the same adapter-facing operations: render position, orientation, resize, disable, click forwarding, and highlights. The composition roots change construction/configuration. State, APIs, and most controller logic should remain untouched.

20.5 Testing strategy

Test layer	Examples	Purpose
Domain unit tests	FEN parsing, UCI application, castling/en-passant/promotion, cloning	Prove deterministic invariants quickly.
State unit tests	start, finish, reset, Memento restore, puzzle index/orientation	Prove aggregate transitions.
Gateway tests	Path, body, query normalization	Protect HTTP contracts without real server.
Controller tests	Optimistic rollback, stale requests, click/drag behavior, server reconciliation	Prove orchestration with fakes.
Server helper tests	ID/UCI/result/history normalization	Protect trust-boundary parsing.
Route integration tests	Real Express + temporary DB	Prove middleware, routing, status codes, transactions.
Browser end-to-end tests	Play game, solve puzzle, network failure	Prove DOM, board library, and server integration.
Live smoke test	GET pages and safe endpoints	Detect deployment contract mismatch.

20.6 Observability and debugging

- **Browser console:** JavaScript exceptions, CSP messages, logs emitted by controllers.
- **Network panel:** exact Request URL, method, status, timing, response body, initiator, cache behavior.
- **Server terminal:** route exceptions, engine/ML failures, external API errors, startup port.
- **curl:** isolates server/API health from frontend JavaScript and browser caching.
- **Database inspection:** confirms persisted moves/result/PGN and helps identify client-server divergence.
- **Correlation IDs (recommended):** attach one request ID to browser log, server log, engine operation, and DB write.

20.7 Scaling roadmap

Current scale	Pressure point	Potential evolution
Single local user/process	No issue	Keep modular monolith; simplest operational model.
More concurrent users	Synchronous DB and engine contention	Engine pool, request queue, DB indexing, rate limits.
Long personalized analysis	HTTP request remains open for many games	Asynchronous job resource: POST job, poll/stream status, persist results.
Multiple server instances	In-memory engine/session coordination	External job queue and shared database/storage.
Large frontend feature set	Controller size and page-level state growth	Introduce use-case command objects or feature subcontrollers only when justified.
Public deployment	Security and abuse	Authentication, CSP, CSRF strategy, input limits, HTTPS, logging, secret management.

21. Design critique, risks, and recommended roadmap

21.1 Strong aspects

- Explicit composition roots make runtime wiring understandable.
- Controller/view/state/gateway boundaries are appropriate for the application size.
- Server authority prevents incomplete frontend legality rules from becoming a competing source of truth.
- Optimistic movement plus Memento rollback produces responsive interaction without sacrificing correctness.

- Generation tokens and cancellation address real async race conditions.
- ChessboardAdapter provides a clean seam around a legacy/global library.
- The same click/drag path avoids divergent behavior between input modes.

21.2 Current risks and debt

Risk	Impact	Recommended action
server.js remains broad	Difficult navigation and mixed reasons to change	Extract routers, services, repositories, and external adapters incrementally.
Incomplete frontend legal validation	Temporary optimistic display of illegal moves	Accept by design or use chess.js client-side solely for prevalidation while retaining server authority.
Puzzle DTO inconsistencies	Boundary normalization and future breakage	Define versioned canonical PuzzleResponse with one moves field.
Duplicate server analysis pipelines	Bug fixes must be duplicated	Extract shared pipeline steps and injected policies.
ML score unit ambiguity	Incorrect threshold/ranking interpretation	Separate score value from score type and model version.
Long synchronous request workflows	Timeouts and blocked capacity	Move analysis to asynchronous jobs.
External CDN dependencies	Offline/CSP/reliability concerns	Vendor/pin jQuery, chessboard.js, and fonts or define CSP deliberately.
Limited integration coverage	Module tests may miss route/static/DOM mismatch	Add route tests and Playwright browser flows.

21.3 Recommended next architectural steps

1. Integrate the hardened game router and remove duplicate inline game handlers.
2. Extract puzzle analysis into a dedicated service module with one canonical DTO normalizer.
3. Add Express integration tests using a temporary SQLite database.
4. Add Playwright tests for initial load, new game, click move, drag move, rejected move, daily puzzle, and source switching.
5. Introduce structured logs with request IDs and elapsed times for engine/Chess.com/ML calls.
6. Decide whether personalized analysis remains synchronous or becomes a queued job API.
7. Document and enforce a Content Security Policy after dependencies are locally hosted or explicitly allowed.

Appendix A - File-by-file responsibility map

File	Responsibility	Should change when...
public/index.html	Play page semantic structure and asset references	Play controls or document structure change
public/puzzles.html	Puzzle page semantic structure	Puzzle controls/metadata/source UI changes
public/css/base.css	Shared tokens, layout, cards, buttons, board basics	Cross-page visual system changes
public/css/play.css	Play-specific presentation	Clocks, moves, log, game banner change
public/css/puzzles.css	Puzzle-specific presentation	Feedback, progress, themes, game list change
js/play/play.mjs	Play composition root	Concrete dependencies or board configuration change
js/play/play-controller.mjs	Play workflows	Sequencing, errors, move/engine behavior change
js/play/play-state.mjs	Play aggregate	Stored client state/invariants change
js/play/play-view.mjs	Play DOM facade	Element binding or rendering changes
js/play/play-api.mjs	Play HTTP gateway	Endpoint path or DTO changes
js/puzzles/puzzles.mjs	Puzzle composition root	Concrete dependencies/board config change

File	Responsibility	Should change when...
js/puzzles/puzzle-controller.mjs	Puzzle workflows	Source, solve, timers, normalization change
js/puzzles/puzzle-state.mjs	Puzzle aggregate	Puzzle client state/invariants change
js/puzzles/puzzle-view.mjs	Puzzle DOM facade	Puzzle rendering/event binding changes
js/puzzles/puzzle-api.mjs	Puzzle HTTP gateway	Puzzle/Chess.com endpoint contracts change
js/shared/chess-position.mjs	Shared FEN/UCI projection domain model	Board metadata or move projection changes
js/shared/chessboard-adapter.mjs	Board-library adapter and click delegation	Board library or highlight behavior changes
js/shared/api-client.mjs	Generic JSON HTTP transport	Headers, parsing, errors, auth, retry policy changes
js/shared/countdown-clock.mjs	Local countdown service	Clock semantics/timing source changes
server.js	Current backend composition and many routes/services	Backend features; ideally delegate to extracted modules
server/game-routes.cjs	Hardened persisted-game router factory	Game HTTP contract or persistence workflow changes
server/game-route-helpers.cjs	Trust-boundary normalization	ID/UCI/result/history syntax changes

Appendix B - Web-development glossary

Term	Meaning in this system
Browser	The client runtime that parses HTML/CSS, runs JavaScript, renders pixels, and sends HTTP requests.
DOM	In-memory object tree created from HTML; JavaScript reads and changes it.
Origin	Scheme + host + port. Ichigo uses http://localhost:8080.
HTTP method	GET retrieves; POST submits a command/body in current Ichigo routes.
Route	A method/path matcher plus a handler function registered in Express.
Middleware	An ordered function that can inspect/modify a request before or after routes.
Static asset	A file served as-is: HTML, CSS, JS, image, chess piece.
Endpoint	A public HTTP interface identified by method and URL pattern.
Request DTO	JSON structure sent to an endpoint.
Response DTO	JSON structure returned by an endpoint.
Promise	A JavaScript object representing a future async completion or failure.
Event loop	Scheduler that executes ready callbacks/promises/timers without one JS thread per connection.
CORS	Browser cross-origin permission mechanism; normally unnecessary for same-origin Ichigo calls.
CSP	Browser policy limiting script/style/resource execution; the observed strict policy was on a 404 page.
FEN	Text snapshot of a chess position and metadata.
UCI move	Coordinate move notation such as e2e4 or e7e8q.
PGN	Portable game notation for a complete chess game and metadata.
Centipawn	Engine evaluation unit equal to one-hundredth of a pawn.

Appendix C - Debugging checklist

- Confirm npm run dev remains running and prints HTTP on 0.0.0.0:8080.
- Confirm window.location.origin is http://localhost:8080.
- Use curl to separate server health from frontend failures.

ICHIGO CHESS | ARCHITECTURE AND DESIGN REPORT

- In DevTools Network, inspect exact URL, method, status, response, and initiator.
- Remember that entering a URL in the address bar always sends GET.
- Treat /favicon.ico 404 as cosmetic unless the missing resource is an application module.
- Check the browser console for the original exception; do not hide cause.message behind a generic network error.
- Verify API client stores a bound fetch implementation.
- After replacing modules, disable cache and hard refresh.
- For state races, inspect inputLocked, sessionVersion, startRequestVersion, loadVersion, and active AbortControllers.
- For client/server divergence, compare browser state with GET /game/:id and returned FEN/history.
- For personalized puzzles, log elapsed time separately for Chess.com, engine evaluation, and Python scoring.

Appendix D - One-page mental model

STARTUP

npm run dev → Node evaluates server.js → Express registers routes → app.listen(8080)

PAGE LOAD

GET / → index.html → CSS/scripts/modules → play.mjs constructs objects → initialize listeners

USER EVENT

click/drag → View/Board callback → Controller workflow → State/Domain mutation → render

SERVER COMMAND

Controller → Gateway → JsonApiClient → fetch → Express route → chess.js/DB/engine → JSON

SUCCESS

JSON → client parses → controller reconciles state → view/board render

FAILURE

HTTP/network/domain error → typed error → rollback or feedback → state remains coherent

CORE DESIGN RULE

Browser owns interaction and projection; server owns legal persisted Play truth.

Conclusion

The larger picture is that Ichigo is not “an HTML file calling server.js.” It is a distributed runtime composed of several processes and multiple layers of code. The browser and server communicate only through explicit HTTP contracts. Within the browser, composition roots construct controllers, views, state aggregates, gateways, and adapters. Within Node, one HTTP listener dispatches each request through ordered Express middleware and route handlers to persistence, chess rules, engine processes, and external services. Understanding those boundaries makes the system easier to debug and safer to extend: change presentation in views/CSS, workflows in controllers, state invariants in domain objects, transport contracts in gateways/routes, and external technology details behind adapters.

Practical architectural test

When adding code, ask: Who owns this decision? Is it presentation, application workflow, domain state, infrastructure transport, or backend integration? Put the code at that boundary and expose the smallest interface needed by the next layer.